МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №9

з дисципліни «Професійний Пайтон»

на тему: «Паралельність, багатопроцесність та оптимізація»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Дослідити підходи до паралельного та конкурентного виконання програм у Python і вплив GIL на CPU-bound та I/O-bound задачі; навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor, знаходити bottlenecks за допомогою timeit/perf_counter, cProfile і tracemalloc, застосовувати NumPy-векторизацію та caching і експериментально визначати, який спосіб оптимізації дійсно прискорює операцію проєкту.

2. Індивідуальне завдання

Варіант 15. Облік результатів наукових експериментів. Предметна область Lab_01: назва експерименту, researcher, параметр, результат і дата; базові можливості збережено. На великому масиві результатів профілювати average, minimum, maximum, standard deviation, range filtering, статистику researcher та числову обробку результатів. Основний акцент — порівняння Python loops, NumPy vectorization і ProcessPoolExecutor; ThreadPoolExecutor використати для паралельного завантаження наборів експериментальних даних. Caching застосувати до повторної статистики незмінного dataset з invalidation після додавання або зміни результатів.

3. Постановка задачі та опис існуючого проєкту

Проєкт experiment-registry після лабораторної №8 містить доменну модель, імпорт/експорт, persistence layer на SQLite і REST API. До нього додано пакет `experiment_registry.performance` з baseline-реалізацією статистики, профілюванням, потоковою та багатопроцесною версіями, NumPy-версією і кешем. Найкращий варіант (NumPy + caching) інтегровано в сервісний шар і API (`GET /experiments/statistics/extended`).

Лістинг 9.1. Нові модулі лабораторної №9

```
performance/
├── __init__.py
├── analytics.py
├── caching.py
├── parallel.py
└── profiling.py
benchmarks/
├── benchmark_performance.py
├── benchmark_search.py
├── benchmark_streaming.py
└── import_experiments.py
```

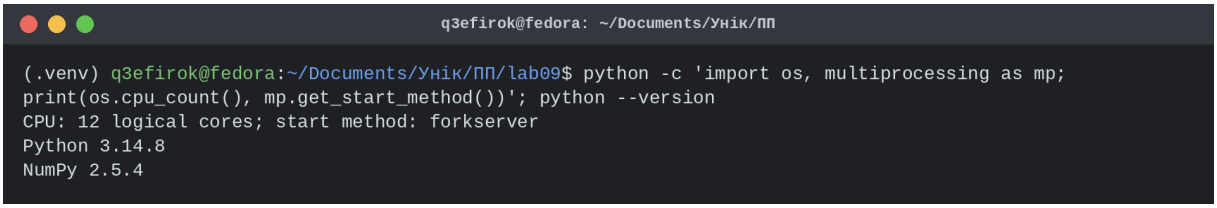


Рис. 9.1. Характеристики середовища вимірювань

3.1. Тестовий dataset

`generate_dataset(size)` створює відтворюваний (seed=15) набір результатів: колонка `researchers` — цілі ідентифікатори 0–7, колонка `results` — значення з нормального розподілу N(300, 25), що імітує вимірювання температури в K. Розміри: 10 000, 100 000 і 1 000 000 записів; для NumPy ті самі дані перетворюються на `float64`/`int64` масиви.

4. Хід виконання

4.1. Baseline implementation

Baseline рахує статистику кількома проходами Python-циклів: середнє, мінімум і максимум, вибіркове стандартне відхилення та кількість викидів (|z| > 3). Окремо реалізовано фільтр діапазону і статистику за дослідниками через словник.

Лістинг 9.2. Baseline на Python-циклах

```
def python_mean(values: Sequence[float]) -> float:
    total = 0.0
    for value in values:
        total += value
    return total / len(values)


def python_std(values: Sequence[float], mean: float) -> float:
    squares = 0.0
    for value in values:
        squares += (value - mean) ** 2
    return math.sqrt(squares / (len(values) - 1))


def python_outliers(values: Sequence[float], mean: float, std: float) -> int:
    count = 0
    for value in values:
        if abs((value - mean) / std) > 3:
            count += 1
    return count


def python_statistics(values: Sequence[float]) -> Statistics:
    """Baseline: several passes over the data with Python loops."""
    if len(values) < 2:
        raise ValueError("at least two values are required")
    mean = python_mean(values)
    minimum, maximum = python_min_max(values)
    std = python_std(values, mean)
    return Statistics(len(values), mean, minimum, maximum, std, python_outliers(values, mean,
std))


def python_researcher_stats(researchers: Sequence[int], values: Sequence[float]) -> dict[int,
tuple[int, float]]:
    """researcher -> (count, mean)."""
    totals: dict[int, list[float]] = {}
    for researcher, value in zip(researchers, values, strict=True):
        entry = totals.setdefault(researcher, [0.0, 0.0])
        entry[0] += 1
        entry[1] += value
    return {key: (int(count), total / count) for key, (count, total) in sorted(totals.items())}
```

4.2. cProfile і вибір bottleneck

Повний аналіз (`analyze_dataset`: статистика, фільтр, статистика за дослідниками) на 1 000 000 результатах профільовано `cProfile.Profile.runcall`, звіт відсортовано за `tottime`.

Лістинг 9.3. Інструменти вимірювання

```
def profile_call(func: Callable[..., Any], *args: Any, top: int = 12,
                 sort: str = "tottime") -> tuple[Any, str]:
    """Run func under cProfile and return (result, formatted statistics)."""
    profiler = cProfile.Profile()
    result = profiler.runcall(func, *args)
    stream = io.StringIO()
    pstats.Stats(profiler, stream=stream).strip_dirs().sort_stats(sort).print_stats(top)
    return result, stream.getvalue()


def peak_memory_mb(func: Callable[..., Any], *args: Any) -> tuple[Any, float]:
    """Peak memory allocated by Python objects during the call (tracemalloc)."""
    tracemalloc.start()
    try:
        result = func(*args)
        _, peak = tracemalloc.get_traced_memory()
    finally:
        tracemalloc.stop()
    return result, peak / 1024 / 1024


def time_runs(func: Callable[[], Any], repeat: int = 5) -> list[float]:
    """Wall-clock time of several runs (seconds)."""
    runs = []
    for _ in range(repeat):
        start = time.perf_counter()
        func()
        runs.append(time.perf_counter() - start)
    return runs
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py profile
cProfile of analyze_dataset() on 1 000 000 results (sorted by tottime):
         2000018 function calls in 0.494 seconds
   Ordered by: internal time
   List reduced from 16 to 10 due to restriction <10>
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.170    0.170    0.238    0.238 analytics.py:94(python_researcher_stats)
        1    0.107    0.107    0.154    0.154 analytics.py:72(python_outliers)
  1000000    0.068    0.000    0.068    0.000 {method 'setdefault' of 'dict' objects}
        1    0.054    0.054    0.054    0.054 analytics.py:65(python_std)
  1000000    0.048    0.000    0.048    0.000 {built-in method builtins.abs}
        1    0.019    0.019    0.019    0.019 analytics.py:90(python_range_filter)
        1    0.014    0.014    0.014    0.014 analytics.py:48(python_mean)
        1    0.011    0.011    0.011    0.011 analytics.py:55(python_min_max)
        1    0.003    0.003    0.494    0.494 analytics.py:125(analyze_dataset)
        1    0.000    0.000    0.234    0.234 analytics.py:80(python_statistics)
```

Рис. 9.2. Результати cProfile

`ncalls` — кількість викликів, `tottime` — час усередині самої функції без вкладених викликів, `cumtime` — разом із вкладеними. Загальний час — 0.494 с. Найбільше часу забирають `python_researcher_stats` (cumtime 0.238 с, з них мільйон викликів `dict.setdefault`) і група функцій статистики: `python_outliers` (0.154 с, мільйон викликів `abs`), `python_std` (0.054 с), середнє та мінімум/максимум. Разом це понад 95% часу. Обидва bottlenecks — CPU-bound: інтерпретатор виконує мільйони елементарних операцій над Python-об’єктами, введення-виведення відсутнє. Основним об’єктом оптимізації обрано функцію

статистики (mean/min/max/std/outliers), статистику за дослідниками оптимізовано додатково.

4.3. tracemalloc і peak memory

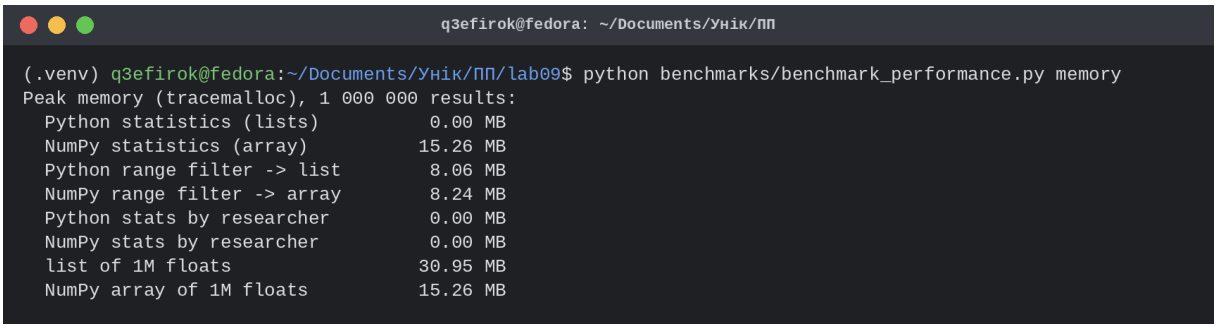


Рис. 9.3. Пікове використання пам’яті

Baseline-статистика не створює нових об’єктів (лише проходить по наявному списку), тому її піковий приріст близький до нуля, а NumPy-версія створює тимчасовий масив `values - mean` (≈15 МБ для 1 000 000 значень). Водночас сам масив NumPy займає вдвічі менше пам’яті, ніж список Python з тих самих чисел (кожен елемент списку — окремий об’єкт float).

4.4. Threaded implementation і ThreadPoolExecutor

Для паралельної версії дані ділиться на частини (`chunk`), кожна частина обробляється одним проходом алгоритму Велфорда (кількість, середнє, M2, мінімум, максимум), а часткові результати зливаються формулою Чана для паралельної дисперсії. Другий раунд рахує викиди за глобальними середнім і відхиленням. Та сама функція `parallel_statistics` працює з будь-яким `Executor` — пулом потоків або процесів.

Лістинг 9.4. Map-reduce статистики для потоків і процесів

```
def partial_moments(values: Sequence[float]) -> Partial:
    """One pass over a chunk (runs inside a worker)."""
    count, mean, m2 = 0, 0.0, 0.0
    minimum = maximum = values[0]
    for value in values:
        count += 1
        delta = value - mean
        mean += delta / count
        m2 += delta * (value - mean)
        if value < minimum:
            minimum = value
        elif value > maximum:
            maximum = value
    return count, mean, m2, minimum, maximum


def merge_moments(parts: Sequence[Partial]) -> Partial:
    """Combine partial results (Chan et al. parallel variance)."""
    count, mean, m2, minimum, maximum = parts[0]
    for n, part_mean, part_m2, part_min, part_max in parts[1:]:
        total = count + n
        delta = part_mean - mean
        mean += delta * n / total
        m2 += part_m2 + delta * delta * count * n / total
        count = total
        minimum, maximum = min(minimum, part_min), max(maximum, part_max)
    return count, mean, m2, minimum, maximum
```

```
def parallel_statistics(values: Sequence[float], executor: Executor, workers: int) ->
Statistics:
    """Map-reduce over chunks with any executor (threads or processes)."""
    chunks = chunk(values, workers)
    count, mean, m2, minimum, maximum = merge_moments(list(executor.map(partial_moments,
chunks)))
    std = math.sqrt(m2 / (count - 1))
    outliers = sum(executor.map(count_outliers, chunks, [mean] * len(chunks), [std] *
len(chunks)))
    return Statistics(count, mean, minimum, maximum, std, outliers)
```

4.5. Synchronization

Спільний змінюваний стан (лічильник прогресу завантаження файлів) захищено `threading.Lock`. Без блокування операція «прочитати — змінити — записати» не є атомарною: потік може бути перемкнений між читанням і записом, і чужі зміни перезаписуються (race condition).

Лістинг 9.5. Lock і демонстрація race condition

```
class ProgressCounter:
    """Shared mutable counter protected by a Lock."""

    def __init__(self) -> None:
        self.value = 0
        self._lock = threading.Lock()

    def increment(self) -> None:
        with self._lock:
            current = self.value
            time.sleep(0)  # give other threads a chance to run between read and write
            self.value = current + 1


class UnsafeCounter:
    """The same read-modify-write without a lock: updates can be lost (race condition)."""

    def __init__(self) -> None:
        self.value = 0

    def increment(self) -> None:
        current = self.value
        time.sleep(0)
        self.value = current + 1


def hammer(counter: "ProgressCounter | UnsafeCounter", threads: int = 8, increments: int = 1000)
-> int:
    def work() -> None:
        for _ in range(increments):
            counter.increment()

    pool = [threading.Thread(target=work) for _ in range(threads)]
    for thread in pool:
        thread.start()
    for thread in pool:
        thread.join()
    return counter.value
```

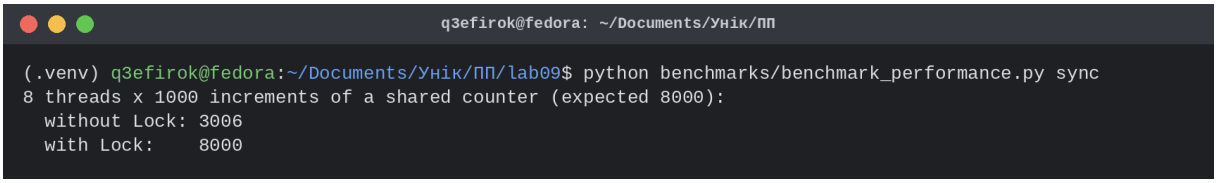


Рис. 9.4. 8 потоків по 1000 інкрементів

Без `Lock` замість 8000 отримано 3006 — більше половини оновлень втрачено; з `Lock` результат завжди правильний.

4.6. ThreadPoolExecutor для завантаження наборів даних (I/O-bound)

Лістинг 9.6. Паралельне завантаження файлів

```
def load_series(path: Path, latency: float = 0.0) -> list[float]:
    """Read one data file; `latency` imitates slow network storage (I/O wait)."""
    if latency:
        time.sleep(latency)
    with path.open(encoding="utf-8", newline="") as file:
        reader = csv.reader(file)
        next(reader)
        return [float(row[1]) for row in reader]


def load_threaded(paths: Sequence[Path], workers: int = 4, latency: float = 0.0,
                  progress: "ProgressCounter | None" = None) -> list[list[float]]:
    def task(path: Path) -> list[float]:
        data = load_series(path, latency)
        if progress is not None:
            progress.increment()
        return data

    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(task, paths))
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py io
Loading 8 files x 50 000 rows; mean of 3 runs
Storage                    | Sequential, s | ThreadPool(4), s | ThreadPool(8), s
local disk                 |         0.105 |      0.108 (1.0x) |      0.107 (1.0x)
network (+0.2 s per file)  |         1.709 |      0.487 (3.5x) |      0.308 (5.5x)
Results of sequential and threaded loading are identical: True
```

Рис. 9.5. Послідовне і паралельне завантаження 8 файлів

Для локального диска (дані в кеші ОС) час визначає розбір CSV, тобто CPU-робота під GIL, тому потоки не допомагають (1.0×). Коли файл надходить із затримкою (імітація мережевого сховища, 0.2 с на файл), потік під час очікування відпускає GIL, і 4 потоки дають 3.5×, а 8 — 5.5× прискорення. Саме для такого I/O-bound сценарію ThreadPoolExecutor і доцільний.

4.7. Multiprocessing і ProcessPoolExecutor

`ProcessPoolExecutor` запускає окремі процеси інтерпретатора (у Python 3.14 на Linux — методом `forkserver`), кожен зі своїм GIL, тож частини даних обробляються справді паралельно. Ціна — серіалізація (pickle) частин списку і результатів між процесами. У benchmark пул створюється один раз і прогрівається до вимірювань, тому час запуску процесів не враховано.

Лістинг 9.7. ProcessPoolExecutor

```
def process_statistics(values: Sequence[float], workers: int = 4) -> Statistics:
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return parallel_statistics(values, executor, workers)


def chunk(values: Sequence[float], parts: int) -> list[list[float]]:
    """Split data into `parts` nearly equal contiguous chunks."""
    size = math.ceil(len(values) / parts)
    return [list(values[i:i + size]) for i in range(0, len(values), size)]
```

4.8. NumPy implementation

Лістинг 9.8. Векторизовані реалізації

```
def numpy_statistics(values: NDArray[np.float64]) -> Statistics:
    """Vectorized: every loop runs in optimized C code inside NumPy."""
    if values.size < 2:
        raise ValueError("at least two values are required")
    mean = float(values.mean())
    std = float(values.std(ddof=1))
    outliers = int(np.count_nonzero(np.abs((values - mean) / std) > 3))
    return Statistics(int(values.size), mean, float(np.min(values)), float(np.max(values)), std,
outliers)


def numpy_range_filter(values: NDArray[np.float64], low: float, high: float) ->
NDArray[np.float64]:
    return values[(values >= low) & (values <= high)]


def numpy_researcher_stats(researchers: NDArray[np.int64], values: NDArray[np.float64]) ->
dict[int, tuple[int, float]]:
    counts = np.bincount(researchers)
    sums = np.bincount(researchers, weights=values)
    return {int(i): (int(counts[i]), float(sums[i] / counts[i])) for i in
np.flatnonzero(counts)}
```

Векторизація замінює цикл інтерпретатора однією операцією над масивом, яку виконує скомпільований код NumPy; `np.bincount` рахує кількість і суму за дослідниками за один прохід.

4.9. Caching implementation

Лістинг 9.9. Кеш статистики з інвалідацією за версією

```
class CachedStatistics:
    """Keeps results in a NumPy array; statistics are recomputed only after a change.

    Every modification increments `version`; the cache key is the version, so a stale
    result can never be returned (cache invalidation).
    """

    def __init__(self, values: Iterable[float]) -> None:
        self._values = np.fromiter(values, dtype=np.float64)
        self.version = 0
        self._cache: dict[int, Statistics] = {}
        self.hits = 0
        self.misses = 0

    def statistics(self) -> Statistics:
        cached = self._cache.get(self.version)
        if cached is not None:
            self.hits += 1
            return cached
        self.misses += 1
        result = numpy_statistics(self._values)
        self._cache = {self.version: result}  # keep only the current version
        return result

    def add(self, value: float) -> None:
        self._values = np.append(self._values, value)
        self.version += 1

    def update(self, index: int, value: float) -> None:
        self._values[index] = value
        self.version += 1

    def __len__(self) -> int:
        return int(self._values.size)
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py cache
Caching of statistics for an unchanged dataset (1 000 000 results):
  first call (miss)           7.042 ms
  repeated call (hit)        0.0008 ms   speedup 9,218x
  after add() -> version 1, recomputed in 4.829 ms; mean 300.0088 -> 300.0185
  after update() -> version 2; hits=7, misses=2
```

Рис. 9.6. Повторне отримання статистики та інвалідація

Повторний виклик для незмінних даних повертає готовий результат приблизно в 9,218 разів швидше. Кожна зміна (`add`, `update`) збільшує `version`, тому застарілий результат повернути неможливо. Той самий принцип застосовано в сервісі: `ExperimentRegistry` збільшує версію при кожному записі в БД, а endpoint `/experiments/statistics/extended` віддає закешовану NumPy-статистику.

Лістинг 9.10. Кешована NumPy-статистика в сервісному шарі

```
    def extended_statistics(self, parameter: str | None = None) -> "ExtendedStatistics":
        """NumPy statistics with caching; the cache is invalidated by any write."""
        key = (parameter, self.version)
        cached = self._stats_cache.get(key)
        if cached is not None:
            self.cache_hits += 1
            return cached
        self.cache_misses += 1
        with session_scope(self._sessions) as session:
            values = np.asarray(ExperimentRepository(session).result_values(parameter),
dtype=np.float64)
        result = ExtendedStatistics.from_values(values, parameter, self.version)
        self._stats_cache[key] = result
        return result
```

5. Експериментальна частина

5.1. Benchmark: 3 розміри dataset, 4 реалізації, 5 повторів

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py benchmark
Bottleneck: statistics (mean, min, max, std, outliers); 5 runs, workers=4
  Dataset | Method     | Workers | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 |  Mean, s | Peak MB | Speedup | OK
   10,000 | Sequential |       1 | 0.0013 | 0.0013 | 0.0013 | 0.0013 | 0.0014 |   0.0013 |    0.00 |   1.00x | True
   10,000 | Threads    |       4 | 0.0018 | 0.0019 | 0.0017 | 0.0017 | 0.0017 |   0.0018 |    0.10 |   0.76x | True
   10,000 | Processes  |       4 | 0.0012 | 0.0012 | 0.0011 | 0.0012 | 0.0012 |   0.0012 |    0.13 |   1.15x | True
   10,000 | NumPy      |       - | 0.0001 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |   0.0000 |    0.15 |  33.20x | True
  100,000 | Sequential |       1 | 0.0133 | 0.0133 | 0.0134 | 0.0137 | 0.0137 |   0.0135 |    0.00 |   1.00x | True
  100,000 | Threads    |       4 | 0.0167 | 0.0162 | 0.0162 | 0.0161 | 0.0172 |   0.0165 |    0.95 |   0.82x | True
  100,000 | Processes  |       4 | 0.0108 | 0.0120 | 0.0121 | 0.0127 | 0.0107 |   0.0117 |    1.03 |   1.16x | True
  100,000 | NumPy      |       - | 0.0007 | 0.0007 | 0.0007 | 0.0007 | 0.0007 |   0.0007 |    1.53 |  18.82x | True
1,000,000 | Sequential |       1 | 0.1279 | 0.1294 | 0.1309 | 0.1329 | 0.1329 |   0.1308 |    0.00 |   1.00x | True
1,000,000 | Threads    |       4 | 0.1439 | 0.1465 | 0.1446 | 0.1451 | 0.1492 |   0.1459 |    9.54 |   0.90x | True
1,000,000 | Processes  |       4 | 0.0973 | 0.1016 | 0.0970 | 0.0985 | 0.0980 |   0.0985 |   10.03 |   1.33x | True
1,000,000 | NumPy      |       - | 0.0060 | 0.0085 | 0.0067 | 0.0059 | 0.0061 |   0.0066 |   15.26 |  19.73x | True
```

Рис. 9.7. Результати benchmark

Таблиця 9.1. Таблиця benchmark (час, с)

Dataset	Method	Workers	Run 1	Run 2	Run 3	Run 4	Run 5	Mean, s	Speedup
10 000	Sequential	1	0.0013	0.0013	0.0013	0.0013	0.0014	0.0013	1.00×
10 000	Threads	4	0.0018	0.0019	0.0017	0.0017	0.0017	0.0018	0.76×
10 000	Processes	4	0.0012	0.0012	0.0011	0.0012	0.0012	0.0012	1.15×
10 000	NumPy	-	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	33.20×
100 000	Sequential	1	0.0133	0.0133	0.0134	0.0137	0.0137	0.0135	1.00×
100 000	Threads	4	0.0167	0.0162	0.0162	0.0161	0.0172	0.0165	0.82×

Dataset	Method	Workers	Run 1	Run 2	Run 3	Run 4	Run 5	Mean, s	Speedup
100 000	Processes	4	0.0108	0.0120	0.0121	0.0127	0.0107	0.0117	1.16×
100 000	NumPy	-	0.0007	0.0007	0.0007	0.0007	0.0007	0.0007	18.82×
1 000 000	Sequential	1	0.1279	0.1294	0.1309	0.1329	0.1329	0.1308	1.00×
1 000 000	Threads	4	0.1439	0.1465	0.1446	0.1451	0.1492	0.1459	0.90×
1 000 000	Processes	4	0.0973	0.1016	0.0970	0.0985	0.0980	0.0985	1.33×
1 000 000	NumPy	-	0.0060	0.0085	0.0067	0.0059	0.0061	0.0066	19.73×

Таблиця 9.2. Таблиця speedup відносно baseline

Dataset	Sequential	Threads (4)	Processes (4)	NumPy
10 000	1.00×	0.76×	1.15×	33.20×
100 000	1.00×	0.82×	1.16×	18.82×
1 000 000	1.00×	0.90×	1.33×	19.73×

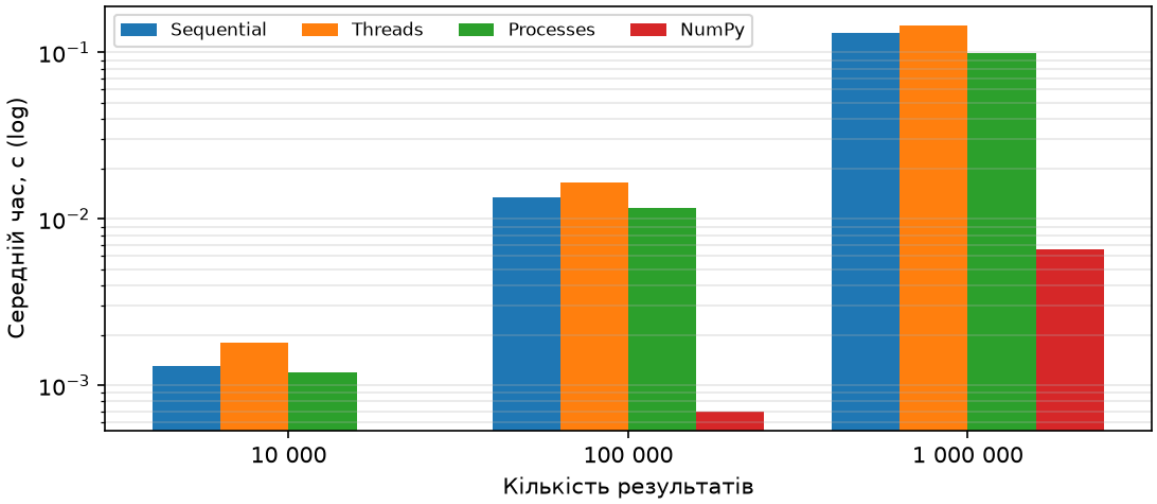


Рис. 9.8. Середній час виконання статистики (логарифмічна шкала)

5.2. Дослідження кількості workers

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py workers
1 000 000 results, sequential baseline 0.1327 s; mean of 3 runs; os.cpu_count()=12
Workers | ThreadPool, s | Speedup | ProcessPool, s | Speedup
      1 |        0.1512 |   0.88x |         0.2555 |   0.52x
      2 |        0.1435 |   0.92x |         0.1474 |   0.90x
      4 |        0.1472 |   0.90x |         0.1020 |   1.30x
      8 |        0.1489 |   0.89x |         0.0853 |   1.56x
     12 |        0.1511 |   0.88x |         0.0736 |   1.80x
```

Рис. 9.9. Вплив кількості workers (1 000 000 результатів)

Таблиця 9.3. Workers: потоки vs процеси

Workers	ThreadPool, с	Speedup	ProcessPool, с	Speedup
1	0.1512	0.88×	0.2555	0.52×
2	0.1435	0.92×	0.1474	0.90×
4	0.1472	0.90×	0.1020	1.30×
8	0.1489	0.89×	0.0853	1.56×
12	0.1511	0.88×	0.0736	1.80×

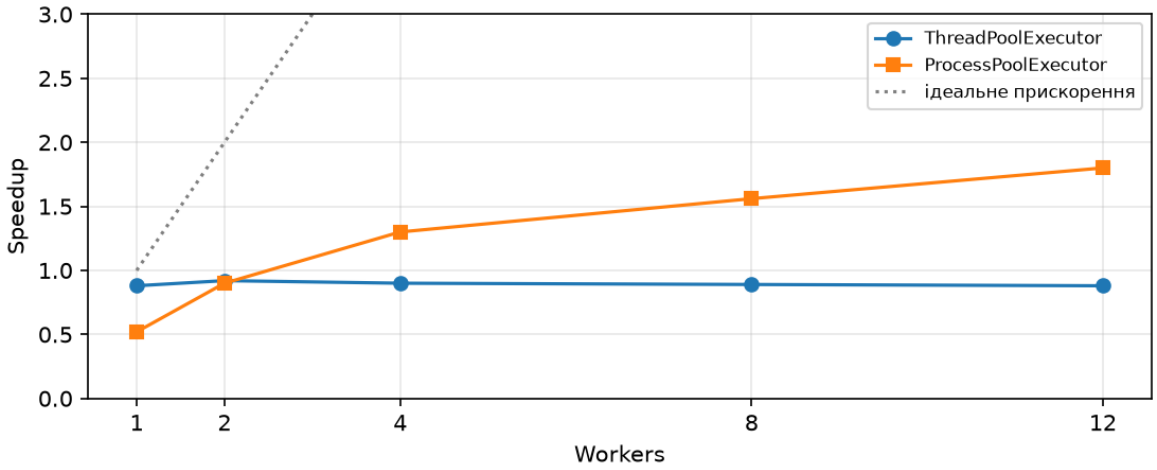


Рис. 9.10. Speedup залежно від кількості workers

5.3. Інші профільовані операції

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab09$ python benchmarks/benchmark_performance.py operations
Other profiled operations, 1 000 000 results, mean of 5 runs:
Operation                | Python, s |  NumPy, s | Speedup | Same result
statistics by researcher |    0.1049 |   0.00185 |   56.8x | True
range filter [250, 350]  |    0.0215 |   0.00118 |   18.1x | True
```

Рис. 9.11. Статистика за дослідниками та фільтр діапазону: Python vs NumPy

5.4. Перевірка правильності

Кожен вимір benchmark перевіряє, що результат збігається з baseline (`Statistics.close_to`, відносна похибка 1e-9) — колонка OK. Окремі тести порівнюють усі реалізації між собою, злиття часткових моментів, розбиття на частини, кеш з інвалідацією, Lock і паралельне завантаження.

Лістинг 9.11. Тести правильності оптимізацій

```
def test_process_statistics_equal_baseline(dataset: Dataset) -> None:
    assert process_statistics(dataset.results, 2).close_to(python_statistics(dataset.results),
rel=1e-9)


def test_merging_partial_moments_equals_single_pass() -> None:
    values = [1.0, 2.0, 4.0, 8.0, 16.0, 32.0, 64.0]
    merged = merge_moments([partial_moments(part) for part in chunk(values, 3)])

    assert merged[0] == len(values)
    assert merged[1:] == pytest.approx(partial_moments(values)[1:])


def test_cache_returns_same_object_until_invalidated() -> None:
    cached = CachedStatistics([1.0, 2.0, 3.0])
    first = cached.statistics()

    assert cached.statistics() is first and (cached.hits, cached.misses) == (1, 1)
    cached.add(10.0)
    assert cached.statistics().maximum == 10.0 and cached.version == 1
    cached.update(0, -5.0)
    assert cached.statistics().minimum == -5.0 and cached.misses == 3 and len(cached) == 4
```

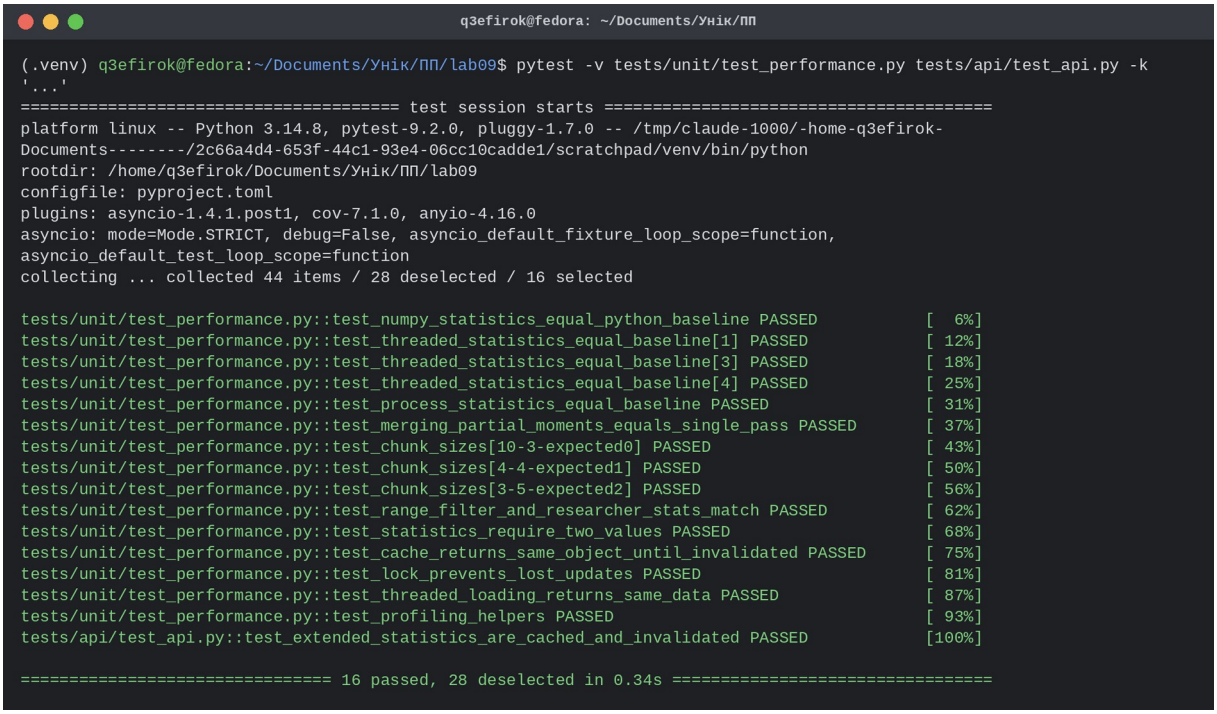


Рис. 9.12. Результати тестів оптимізацій

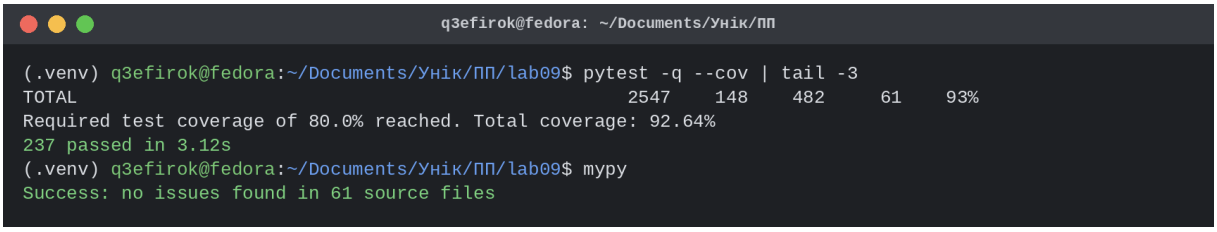


Рис. 9.13. Увесь test suite і mypy

5.5. Аналіз результатів

Таблиця 9.4. Висновки експерименту

Питання	Відповідь за результатами
Яка функція — bottleneck?	Статистика за дослідниками та статистика (особливо пошук викидів і std) на Python-циклах.
Яка частка часу?	Разом понад 95% із 0.494 с повного аналізу; статистика — близько половини.
CPU-bound чи I/O-bound?	CPU-bound: мільйони операцій інтерпретатора, без очікування введення-виведення.
Чи допоміг ThreadPoolExecutor?	Ні: 0.90× на 1 000 000 — GIL дозволяє виконувати байткод лише одному потоку; є ще накладні витрати.
Чи допоміг ProcessPoolExecutor?	Помірно: 1.33× при 4 workers, до 1.80× при 12; ліміт — pickle даних між процесами.
Оптимальна кількість workers	Для процесів — 12 (найкращий speedup), але приріст після 4 малий; для потоків кількість не впливає.
Чи дав перевагу NumPy?	Так, найбільшу: 19.73× на 1 000 000; на всіх розмірах — 19–33×.
Як змінилася пам’ять?	NumPy-масив удвічі компактніший за список, але створює тимчасові масиви (≈15 МБ).
Чи дав caching прискорення?	Так, повторний виклик ≈9,218× швидший; після зміни даних — перерахунок.
Найкраща реалізація	NumPy + caching; паралельність потрібна лише для I/O (завантаження файлів).
Чи однаковий результат для малих і великих?	Ні: на 10 000 записів процеси майже не виграють, а потоки повільніші; переваги ростуть з обсягом.
Що лишити в production?	NumPy-статистику з кешем і інвалідацією (інтегровано в API) та ThreadPoolExecutor для I/O.

Реальне прискорення процесів (1.33× при 4 workers замість ідеальних 4×) нижче ідеального через закон Амдала і накладні витрати: частини списку серіалізуються й передаються в інші процеси двічі (для моментів і для викидів), результати повертаються назад, а частина роботи (розбиття, злиття) виконується послідовно. Обробка одного елемента дуже дешева, тому вартість передачі даних порівняння з обчисленнями. NumPy виграє, бо прибирає саму причину повільності — інтерпретацію циклу для кожного елемента.

5.6. Завдання підвищеної складності

- паралельна редукція з алгоритмом Велфорда і формулою Чана (стійка до похибок дисперсія);
- спільна функція map-reduce для потоків і процесів через абстракцію `Executor`;
- дослідження до 12 workers і порівняння I/O-bound та CPU-bound сценаріїв;
- кеш з інвалідацією за версією, інтегрований у сервіс і REST API;
- збереження результатів benchmark у CSV і побудова графіків часу та speedup.

6. Відповіді на контрольні запитання

1. Що таке concurrency?

Concurrency (конкурентність) - це організація виконання кількох задач так, щоб їхнє виконання перекривалося в часі: програма перемикається між задачами, наприклад, поки одна чекає на I/O, виконується інша. Задачі не обов'язково виконуються фізично одночасно, тому concurrency можлива навіть на одному ядрі CPU.

2. Що таке parallelism?

Parallelism (паралелізм) - це фізично одночасне виконання кількох обчислень на різних ядрах CPU. У CPython для паралельного виконання Python-коду зазвичай використовують кілька процесів (multiprocessing, ProcessPoolExecutor), оскільки кожен процес має власний GIL.

3. Чим concurrency відрізняється від parallelism?

Concurrency описує структуру програми, у якій кілька задач чергуються та перекриваються в часі, а parallelism - справді одночасне виконання на кількох ядрах. Паралелізм є окремим випадком конкурентності: потоки в CPython дають concurrency для I/O-bound задач, а процеси дають справжній parallelism для CPU-bound обчислень.

4. Що таке process?

Process - це екземпляр програми, що виконується, з власним ізольованим адресним простором пам'яті, ресурсами ОС і щонайменше одним потоком. У Python кожен процес має окремий інтерпретатор і власний GIL, тому процеси можуть паралельно виконувати Python-код на різних ядрах.

5. Що таке thread?

Thread - це потік виконання всередині процесу, тобто послідовність інструкцій, яку незалежно планує операційна система. Усі потоки одного процесу спільно використовують його пам'ять і ресурси, тому обмін даними між ними простий, але потребує синхронізації.

6. Чим process відрізняється від thread?

Процеси мають ізольовану пам'ять, а потоки одного процесу ділять спільну пам'ять. Створення процесу дорожче, а обмін даними між процесами потребує serialization (pickle) та IPC, зате процеси в CPython обходять GIL; потоки легші, але для CPU-bound Python-коду обмежені GIL.

7. Що таке GIL?

GIL (Global Interpreter Lock) - глобальне блокування інтерпретатора CPython, яке у стандартній збірці дозволяє лише одному потоку одночасно виконувати Python bytecode в межах одного процесу. Через нього велика кількість threads не прискорює CPU-bound Python-код.

8. Яку роль GIL відіграє у CPython?

GIL захищає внутрішні структури інтерпретатора, зокрема лічильники посилань для керування пам'яттю, і спрощує реалізацію CPython та C-розширень. Наслідок: потоки не прискорюють CPU-bound Python-код, але GIL звільняється під час блокуючих I/O-операцій і в багатьох C-функціях (наприклад, NumPy), тому для I/O-bound задач потоки ефективні.

9. Що таке CPU-bound задача?

CPU-bound задача - це задача, час виконання якої визначається переважно обчисленнями процесора, а не очікуванням зовнішніх ресурсів. Прискорити її можна зменшенням кількості операцій, векторизацією (NumPy) або розподілом обчислень між кількома процесами.

10. Наведіть приклад CPU-bound задачі.

У проєкті experiment_registry це обчислення статистики (average, minimum, maximum, standard deviation, range filtering, статистика за researcher) по 1 000 000 результатів експериментів у Python-циклах. Інші типові приклади: пошук простих чисел, обробка зображень, чисельне моделювання.

11. Що таке I/O-bound задача?

I/O-bound задача - це задача, яка більшу частину часу очікує завершення операцій введення-виведення: читання або запису файлів, запитів до бази даних, мережевих запитів чи звернень до зовнішніх API. Процесор у цей час простоює, тому прискорення дає перекриття очікувань за допомогою threads або asyncio.

12. Наведіть приклад I/O-bound задачі.

У проєкті це паралельне завантаження кількох файлів наборів експериментальних даних через ThreadPoolExecutor: більшість часу кожен потік чекає на читання з диска. Інші приклади: HTTP-запити до зовнішнього API, запити до бази даних, завантаження файлів із мережі.

13. Коли доцільно застосовувати threading?

Threading доцільно застосовувати для I/O-bound задач (читання файлів, мережеві запити, звернення до БД), коли потоки більшість часу чекають, а GIL під час очікування звільняється. Також потоки підходять для фонових задач і виклику C-коду, що звільняє GIL; для чистого CPU-bound Python-коду прискорення вони не дають.

14. Коли доцільно застосовувати multiprocessing?

Multiprocessing доцільний для CPU-bound задач, які можна розбити на незалежні частини з достатнім обсягом обчислень, щоб окупити витрати на запуск процесів і передачу даних. У проєкті ProcessPoolExecutor обробляє масив результатів експериментів, розділений на chunks, використовуючи кілька з 12 логічних CPU.

15. Для чого використовується Thread?

Клас threading.Thread використовується для створення окремого потоку виконання: йому передають функцію target та її аргументи args, які виконуються в новому потоці. Після створення потік запускають методом start(), а його завершення очікують методом join().

16. Для чого використовується start()?

Метод start() запускає потік (або процес у multiprocessing.Process): створюється новий потік виконання, у якому викликається метод run(), тобто функція target. Його можна викликати лише один раз для об'єкта; прямий виклик run() виконав би функцію в поточному потоці без жодної паралельності.

17. Для чого використовується join()?

Метод join() блокує поточний потік, доки потік або процес, для якого його викликано, не завершиться; можна задати timeout. Його використовують, щоб дочекатися завершення всіх workers перед обробкою результатів або виходом із програми.

18. Що таке race condition?

Race condition - це помилка, за якої результат залежить від непередбачуваного порядку виконання потоків, що одночасно звертаються до спільних даних, і хоча б один із них їх змінює. Наприклад, counter += 1 складається з читання, збільшення і запису, тому два потоки можуть прочитати те саме значення, і одне оновлення буде втрачено.

19. Що таке critical section?

Critical section (критична секція) - це ділянка коду, яка звертається до спільного змінюваного ресурсу і тому повинна виконуватися не більше ніж одним потоком одночасно. Її захищають примітивами синхронізації, наприклад блоком with lock:, як у проєкті при збільшенні спільного лічильника.

20. Для чого використовується Lock?

threading.Lock - це примітив взаємного виключення (mutex): потік захоплює його через acquire() або with lock:, а інші потоки чекають, доки він не викличе release(). У

проєкті Lock захищає спільний лічильник, який збільшують кілька потоків, щоб уникнути race condition і втрати оновлень.

21. Що таке synchronization?

Synchronization - це координація роботи потоків або процесів для коректного доступу до спільних ресурсів і правильного порядку дій. Засоби синхронізації в Python: Lock, RLock, Semaphore, Event, Condition, потокобезпечна Queue, а також join() для очікування завершення.

22. Чи завжди synchronization покращує performance?

Ні, synchronization забезпечує коректність, а не швидкість: захоплення блокувань має накладні витрати, а потоки, що чекають на Lock, простоюють, тобто захищена частина коду стає послідовною. Надмірна синхронізація може навіть сповільнити програму, тому критичні секції мають бути якомога коротшими.

23. Що таке ThreadPoolExecutor?

ThreadPoolExecutor - клас із модуля concurrent.futures, що керує пулом повторно використовуваних потоків і виконує в них задачі через submit() (повертає Future) або map() (повертає результати). Використання в блоці with гарантує коректне завершення пулу через shutdown().

24. Коли його доцільно використовувати?

ThreadPoolExecutor доцільний для I/O-bound задач із багатьма незалежними операціями очікування: читання кількох файлів, HTTP-запити, звернення до БД. У проєкті ним паралельно завантажуються кілька файлів наборів експериментальних даних, тоді як для CPU-bound статистики він прискорення не дає через GIL.

25. Що таке ProcessPoolExecutor?

ProcessPoolExecutor - клас із модуля concurrent.futures, що керує пулом процесів-workers і виконує в них функції, передаючи аргументи й результати через serialization (pickle). Оскільки кожен процес має власний інтерпретатор і GIL, він забезпечує справжній parallelism.

26. Коли його доцільно використовувати?

ProcessPoolExecutor доцільний для CPU-bound задач, які розбиваються на незалежні великі частини, коли обсяг обчислень значно перевищує витрати на створення процесів і передачу даних. У проєкті результати експериментів ділилися на chunks, кожен chunk обробляв окремий процес, а часткові результати потім об'єднувалися; функція має бути визначена на рівні модуля, а запуск - під if __name__ == "__main__".

27. Для чого використовується executor.map()?

executor.map(func, iterable) застосовує функцію до кожного елемента вхідних даних у workers пулу і повертає ітератор результатів у тому самому порядку, що й вхідні дані. Для ProcessPoolExecutor параметр chunksize групує елементи в пакети, зменшуючи overhead передачі, а виняток із worker повторно піднімається під час отримання відповідного результату.

28. Що таке worker?

Worker - це потік або процес у пулі, який отримує задачі з внутрішньої черги та виконує їх. Кількість workers задається параметром max_workers, наприклад ProcessPoolExecutor(max_workers=4).

29. Чи завжди більша кількість workers дає більшу швидкість?

Ні. Для CPU-bound задач workers понад кількість ядер (у проєкті 12 логічних CPU) не прискорюють обчислення, а лише додають overhead на створення процесів, перемикання контексту та serialization; для потоків у CPU-bound коді прискорення взагалі обмежує GIL. Оптимальну кількість workers визначають експериментально.

30. Що таке multiprocessing overhead?

Multiprocessing overhead - це додаткові витрати часу й пам'яті, що не є корисною роботою: запуск процесів та імпорт модулів у них, serialization аргументів і результатів, передача даних між процесами (IPC) та збирання результатів. Якщо обсяг роботи малий, цей overhead перевищує виграш від паралельності.

31. Що таке serialization?

Serialization - це перетворення об'єкта на послідовність байтів для збереження або передачі, а deserialization - відновлення об'єкта з байтів. У multiprocessing та ProcessPoolExecutor аргументи й результати серіалізуються модулем pickle, тому вони мають бути picklable, а передача великих списків займає помітний час.

32. Чому multiprocessing може бути повільним для маленьких задач?

Тому що фіксовані витрати на запуск процесів, pickle-серіалізацію даних, їх передачу та збирання результатів можуть бути більшими за сам час обчислень. Наприклад, обчислення середнього для 10 000 значень триває мілісекунди, тому ProcessPoolExecutor на такому обсязі дає speedup менше 1.

33. Для чого використовується timeit?

Модуль timeit вимірює час виконання невеликих фрагментів коду або функцій, багаторазово виконуючи їх (параметр number), використовуючи perf_counter і вимикаючи збирач сміття на час заміру. Функція timeit.repeat() виконує кілька серій вимірювань, з яких зазвичай беруть мінімум або медіану.

34. Для чого використовується perf_counter()?

time.perf_counter() повертає значення монотонного таймера з найвищою доступною роздільною здатністю в секундах; змістовною є лише різниця двох викликів. Його використовують для вимірювання тривалості більших операцій: start = perf_counter(), виклик функції, elapsed = perf_counter() - start.

35. Навіщо виконувати декілька benchmark repeats?

Окремий замір нестабільний через шум: фонові процеси ОС, прогрів кешів, збирання сміття, зміну частоти CPU. Кілька повторів дозволяють отримати стабільну оцінку (мінімум, медіану або середнє зі стандартним відхиленням) і зробити обґрунтований висновок про speedup.

36. Що таке profiling?

Profiling - це вимірювання поведінки програми під час виконання: скільки разів викликається кожна функція і скільки часу чи пам'яті вона використовує. Мета профілювання - знайти bottlenecks на основі даних, а не припущень.

37. Для чого використовується cProfile?

cProfile - вбудований детермінований профайлер CPython, який для кожної функції збирає кількість викликів, власний і сукупний час виконання. Його запускають як python -m cProfile -s cumtime script.py або програмно через cProfile.run() зі збереженням у файл .prof, який аналізують модулем pstats.

38. Що показує ncalls?

ncalls показує кількість викликів функції; запис виду 12/3 означає загальну кількість викликів і кількість примітивних (нерекурсивних) викликів. Дуже велике ncalls часто вказує, що функцію викликають у циклі для кожного елемента даних.

39. Що показує tottime?

tottime показує загальний час, проведений безпосередньо в тілі функції, без урахування часу викликаних нею підфункцій. Велике tottime означає, що саме ця функція виконує важкі обчислення.

40. Що показує cumtime?

cumtime показує сукупний час виконання функції разом з усіма підфункціями, які вона викликала. Сортування за cumtime допомагає знайти високорівневу операцію, всередині якої витрачається найбільше часу, а далі за tottime - конкретну функцію-винуватця.

41. Що таке bottleneck?

Bottleneck (вузьке місце) - частина програми, яка обмежує загальну продуктивність, тобто на неї припадає найбільша частка часу виконання або використання ресурсу. Доки bottleneck не усунуто, оптимізація інших частин майже не змінює загального часу.

42. Як знайти bottleneck?

Спочатку вимірюють baseline (perf_counter, timeit) на реалістичному обсязі даних, потім запускають cProfile і сортують результати за cumtime та tottime, а використання пам'яті аналізують через tracemalloc. У проєкті так перевірялося, які саме обчислення статистики (average, standard deviation, range filtering, статистика за researcher) займають найбільше часу на великих наборах результатів.

43. Для чого використовується tracemalloc?

tracemalloc - вбудований модуль для відстеження виділення пам'яті Python-об'єктами: після tracemalloc.start() функція get_traced_memory() повертає поточний і піковий обсяг, а take_snapshot().statistics('lineno') показує рядки коду, що виділяють найбільше пам'яті. У проєкті ним вимірювалася пам'ять під час обчислення статистики на великих наборах даних.

44. Що таке peak memory?

Peak memory - максимальний обсяг пам'яті, виділений одночасно за час спостереження; у tracemalloc це друге значення, яке повертає get_traced_memory(). Він важливий, бо навіть якщо пам'ять згодом звільнено, програмі потрібно досить RAM для піку, наприклад під час створення великих проміжних списків.

45. Що таке NumPy?

NumPy - бібліотека для числових обчислень у Python, основою якої є ndarray - багатовимірний масив однотипних елементів у неперервній пам'яті. Операції над масивами реалізовані на C, тому mean(), min(), max() і std() у проєкті виконуються значно швидше за Python-цикли.

46. Що таке vectorization?

Vectorization - запис обчислень як операцій над цілими масивами замість явних Python-циклів по елементах, наприклад array[(array >= low) & (array <= high)] для range filtering або ((array - array.mean()) ** 2).mean() для дисперсії. Цикл при цьому виконується всередині скомпільованого коду NumPy.

47. Чому NumPy часто швидший за Python loops?

Python-цикл для кожного елемента інтерпретує bytecode, динамічно перевіряє типи й працює з окремими об'єктами float у купі. NumPy зберігає числа компактно в неперервному масиві фіксованого типу і обробляє їх скомпільованим C-кодом з ефективним використанням кешу процесора та SIMD-інструкцій; проте для малих масивів витрати на перетворення списку в масив можуть з'їсти виграш.

48. Що таке caching?

Caching - збереження результату дорогої операції, щоб при повторному запиті з тими самими вхідними даними повернути готовий результат замість повторного обчислення. У проєкті кешується статистика для незмінного набору результатів експериментів.

49. Для чого використовується @cache?

Декоратор functools.cache (з Python 3.9) запам'ятовує результати функції для кожного набору аргументів у необмеженому словнику, тобто є еквівалентом lru_cache(maxsize=None). Аргументи мають бути hashable, а кеш може необмежено зростати, тому його застосовують для чистих функцій з обмеженою кількістю різних аргументів.

50. Для чого використовується @lru_cache?

Декоратор functools.lru_cache(maxsize=N) виконує мемоізацію з обмеженим розміром кешу: коли кеш заповнено, видаляється запис, який найдовше не використовувався (Least Recently Used). Він надає cache_info() зі статистикою hits і misses та cache_clear() для очищення кешу.

51. Що таке cache hit?

Cache hit - ситуація, коли результат для запитаного ключа вже є в кеші й повертається без повторного обчислення. Висока частка hits означає, що кешування ефективне; для lru_cache їх кількість показує cache_info().hits.

52. Що таке cache miss?

Cache miss - ситуація, коли результату для запитаного ключа в кеші немає, тому функція виконує обчислення і зберігає результат у кеші. Перший виклик з новими аргументами завжди є miss, як і перший виклик після invalidation.

53. Що таке cache invalidation?

Cache invalidation - видалення або позначення застарілими записів кешу, коли змінилися дані, від яких залежить результат. У проєкті після додавання або оновлення результату експерименту збільшується лічильник версії (version counter), що входить у ключ кешу, тому стара статистика більше не використовується.

54. Коли кешування може бути небезпечним?

Кешування небезпечне, коли результат залежить від змінюваних даних або зовнішнього стану (база даних, файли, час, випадковість) і немає коректної invalidation - тоді повертаються застарілі дані. Також ризиковані необмежений кеш, що призводить до зростання пам'яті, і кешування змінюваних об'єктів, які викликач може модифікувати.

55. Як обчислюється speedup?

Speedup = T_baseline / T_optimized, де обидва часи виміряні на однакових даних і в однакових умовах, бажано як медіана або мінімум кількох повторів. Наприклад, 8 с / 2 с = 4, тобто прискорення в 4 рази.

56. Що означає speedup менше 1?

Speedup менше 1 означає, що оптимізована версія повільніша за baseline, тобто «оптимізація» погіршила продуктивність. Наприклад, ThreadPoolExecutor для CPU-bound статистики або ProcessPoolExecutor на 10 000 записів можуть дати speedup менше 1 через GIL та overhead.

57. Чому threaded CPU-bound version може бути повільнішою?

Через GIL лише один потік одночасно виконує Python bytecode, тому обчислення фактично залишаються послідовними. Додатково з'являються витрати на створення потоків, перемикання між ними, конкуренцію за GIL і керування пулом, тож загальний час може перевищити послідовну версію.

58. Чому ProcessPoolExecutor може прискорити CPU-bound код?

Кожен процес має власний інтерпретатор і власний GIL, тому кілька процесів виконують Python-код фізично паралельно на різних ядрах CPU. Якщо дані поділено на великі chunks і обчислення значно дорожчі за передачу даних, загальний час зменшується, хоча speedup не перевищує кількості ядер.

59. Чому необхідно перевіряти правильність optimized implementation?

Оптимізація змінює алгоритм або порядок обчислень і може внести помилки: втрачені chunks, неправильне об'єднання часткових результатів, race condition, застарілий кеш. Швидкий, але неправильний результат не має цінності, тому результати порівнюють із baseline, а для float - з допуском (math.isclose, numpy.allclose), бо інший порядок додавання змінює похибку округлення.

60. Чому оптимізацію потрібно починати з profiling?

Інтуїція щодо повільних місць часто помилкова, а оптимізація частини, яка не є bottleneck, майже не змінює загального часу, проте ускладнює код. Profiling показує, де реально витрачаються час і пам'ять, допомагає визначити тип задачі (CPU-bound чи I/O-bound) і вибрати відповідний метод, а baseline-вимір потрібен для обчислення speedup.

7. Висновок

Профілювання показало, що bottlenecks проєкту — статистика за дослідниками та обчислення середнього, стандартного відхилення і викидів Python-циклами; задача CPU-bound. ThreadPoolExecutor не прискорив її через GIL (0.90×), ProcessPoolExecutor дав 1.33× при 4 workers і до 1.80× при 12 workers, бо значну частину часу займає передавання даних між процесами. Найкращий результат дала NumPy-векторизація: 19.73× на 1 000 000 результатів при однакових значеннях.

Потоки виправдані для I/O-bound завантаження файлів (до 5.5× при мережевій затримці), а Lock обов’язковий для спільного стану (без нього втрачено 4994 з 8000 оновлень). Caching прискорив повторну статистику незмінних даних приблизно в 9,218 разів. У production-версії проєкту залишено NumPy-статистику з кешем і інвалідацією; висновок експерименту — паралельність без профілювання не гарантує прискорення, оптимальний метод залежить від характеру bottleneck.